

Compact Instruction Set Extensions for Kyber

Lu Li¹, Student Member, IEEE, Guofeng Qin¹, Yang Yu¹, and Weijia Wang¹, Member, IEEE

Abstract—Kyber is the only post-quantum cryptography (PQC) key encapsulation mechanism in the National Institute of Standards and Technology PQC project. This brief investigates the design of compact instruction set extensions (ISEs) for Kyber. We focus on implementing number-theoretic transform (NTT) and propose a hardware design of the modular multiplication based on an optimized k^2 -reduction. Compared to other works, our design is more compact since the optimized k^2 -reduction comprises multiplications with significantly smaller multipliers than Montgomery reduction and Barrett reduction. Then, we integrate the k^2 -reduction into an instruction for the butterfly transformation. We also propose auxiliary instructions that can switch the half words between two registers to facilitate the rearranging coefficients in NTT. To showcase the advantage of the instructions, we implement the ISEs in a chip design for the Hummingbird E203 core. Compared to the software implementation on RISC-V with assembly code, our co-design implementations for NTT show a speedup by a factor of 2.6. Besides, the area overhead is 93 LUTs and 1 DSP without any additional resources of FFs and RAMs using Artix-7 FPGA, which is more compact than previous software–hardware co-designs of Kyber.

Index Terms—Instruction set extensions (ISEs), Kyber, number-theoretic transform (NTT), post-quantum cryptography (PQC), RISC-V.

I. INTRODUCTION

Kyber is a lattice-based key encapsulation mechanism (KEM) [1], and it is one of the candidates to be standardized in the post-quantum cryptography (PQC) standardization competition by the National Institute of Standards and Technology (NIST) [2]. An important aspect of PQC research is the efficient and secure implementation of these algorithms. Software–hardware co-design is the simultaneous design of hardware and software to implement the desired function, reaching a good implementation tradeoff with high speed and low silicon area cost. RISC-V is a promising open standard instruction set architecture (ISA) based on established RISC principles [3]. It is

Manuscript received 25 January 2023; revised 21 June 2023; accepted 5 October 2023. Date of publication 24 October 2023; date of current version 21 February 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFA1000600; in part by the National Natural Science Foundation of China under Grant 62002202 and Grant 62102216; in part by the Program of Taishan Young Scholars of the Shandong Province under Grant tsqn202306002; in part by the Program of Qilu Young Scholars of Shandong University under Grant 61580082063088; in part by the Mathematical Tianyuan Fund of the National Natural Science Foundation of China under Grant 12226006; in part by the Department of Science and Technology of Shandong Province under Grant SYS202201; and in part by Quan Cheng Laboratory under Grant QCLZD202306. This article was recommended by Associate Editor Y. Jin. (Corresponding author: Weijia Wang.)

Lu Li and Guofeng Qin are with the School of Cyber Science and Technology, Shandong University, Qingdao 266237, China, and also with Quan Cheng Laboratory, Jinan 250014, China (e-mail: lulisdu@mail.sdu.edu.cn; guofengqin@mail.sdu.edu.cn).

Yang Yu is with the Institute for Advanced Study, Tsinghua University, Beijing 100190, China, also with Zhongguancun Laboratory, Beijing 100194, China, and also with National Financial Cryptography Research Center, Beijing, China (e-mail: yu-yang@mail.tsinghua.edu.cn).

Weijia Wang is with the School of Cyber Science and Technology and the Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Qingdao 266237, China, and also with Quan Cheng Laboratory, Jinan 250014, China (e-mail: wjwang@sdu.edu.cn).

Digital Object Identifier 10.1109/TCAD.2023.3327104

an ideal candidate for software–hardware co-design of cryptographic algorithms.

Known works devoted to hardware accelerators for the software–hardware co-design fall into two categories: 1) loosely coupled [4], [5] and 2) tightly coupled [6], [7]. Compared with the loosely coupled strategy, tightly coupled instruction set extensions (ISEs) can deeply reuse the processor resources to achieve lower silicon area overhead. Alkim et al. [6] presented and evaluated a custom extension to the RISC-V instruction set to accelerate polynomial multiplications in Kyber and NewHope. Meanwhile, Fritzmann et al. [7] presented RISQ-V, a set of high-performance and tightly coupled accelerators.

This brief focuses on the tightly coupled design and investigates ISEs for Kyber’s number-theoretic transform (NTT) with a good tradeoff between speed and silicon area overhead. Although previous works [6], [7], [8] pointed out that polynomial generation and polynomial arithmetic are bottlenecks, we only focus on implementing NTT (concretely, the butterfly transformation) in this brief.

We have developed a set of tightly coupled accelerators for modular arithmetic and NTT computations and make them accessible through custom instructions. Furthermore, we have used these custom instructions to accelerate the implementation of Kyber. The detailed contributions are threefold.

- 1) We adopt the optimized k^2 -reduction to the design of modulo operation that comprises multiplications with significantly smaller multipliers and reduces the hardware cost.
- 2) We utilize the optimized k^2 -reduction in the compact ISEs for Kyber’s polynomial arithmetic and introduce a custom butterfly transformation. Besides, we propose swap instructions to simplify and accelerate the management of coefficients in the NTT.
- 3) We implement the new ISEs in a chip design for the Hummingbird E203 core. Compared to the software implementation on RISC-V with assembly code, our co-design implementations achieve a significant tradeoff between speed and hardware overhead.

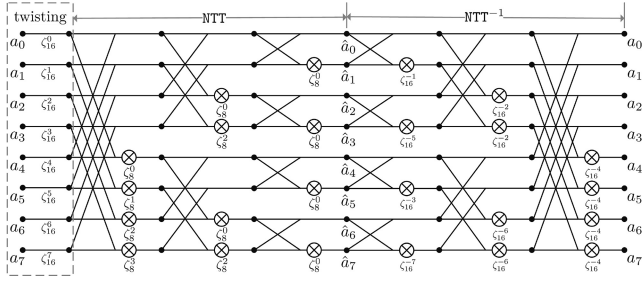
Compared with existing designs of tightly coupled accelerators [6], [7], performance gains using our ISEs lie between [6] and [7], and the resource overhead is smaller than the two works. The implementation of this work is publicly available at <https://github.com/wjwangcrypto/KyberISE>.

Section II briefly introduces the operations related to our implementation. Section III compares different reductions in integrated circuit cost, confirming k^2 -reduction as the basis for the custom modular arithmetic, and describes the ISE in detail. The experimental results are presented in Section IV. Section V concludes this brief.

II. BACKGROUND

A. Number-Theoretic Transform

The NTT is a variant of the discrete Fourier transform (DFT) defined over finite fields. The cyclic forward-NTT (NTT) computes the Chinese remainder map: $\mathbb{Z}_q[X]/(X^n - 1) \mapsto \prod_i \mathbb{Z}_q[X]/(X - \zeta_n^i)$, $i = 0, 1, 2, \dots, n-1$, ζ_n is a primitive n th root of unity. The inverse-NTT (NTT^{-1}) maps integers from the NTT domain to

Fig. 1. NTT and NTT^{-1} with GS-butterfly and twisting over $\mathbb{Z}_q/(X^8 + 1)$.**Algorithm 1** k^2 -reduction

Input: $C \in (0, q^2)$, where $q = k \cdot 2^m + 1$ is a prime; $l = \lceil \log_2 q \rceil$, $k_0 = \delta k \cdot 2^i$, $k_1 = \delta k \cdot 2^j$, where $0 \leq i, j \leq m$ and $\delta \in \{-1, 1\}$

Output: $r = k_0 k_1 C \bmod q$

```

1:  $C_{01} \leftarrow \lfloor C/2^{m-i} \rfloor$ ;  $\triangleright C_{01}: (2l - m + i)_{\text{bit}}$ 
2:  $C_{00} \leftarrow C \bmod 2^{m-i}$ ;  $\triangleright C_{00}: (m - i)_{\text{bit}}$ 
3:  $t_1 \leftarrow k_0 \cdot C_{00}$ ;  $\triangleright t_1: l_{\text{bit}}$ 
4:  $t_2 \leftarrow t_1 - \delta C_{01}$ ;  $\triangleright t_2: \max((2l - m + i)_{\text{bit}}, l_{\text{bit}})$ 
5:  $C_{11} \leftarrow \lfloor t_2/2^{m-j} \rfloor$ ;  $\triangleright C_{11}: (\lceil \log_2 t_2 \rceil - m + j)_{\text{bit}}$ 
6:  $C_{10} \leftarrow t_2 \bmod 2^{m-j}$ ;  $\triangleright C_{10}: (m - j)_{\text{bit}}$ 
7:  $t_3 \leftarrow k_1 \cdot C_{10}$ ;  $\triangleright t_3: l_{\text{bit}}$ 
8:  $r \leftarrow t_3 - \delta C_{11}$ ;  $\triangleright r: \max((\lceil \log_2 t_2 \rceil - m + j)_{\text{bit}}, l_{\text{bit}})$ 

```

the original one, i.e.,

$$\hat{\mathbf{a}} = \text{NTT}(\mathbf{a}) = \sum_{i=0}^{n-1} \hat{a}_i X^i \text{ with } \hat{a}_i = \sum_{j=0}^{n-1} a_j \zeta_n^{ij}$$

$$\mathbf{a} = \text{NTT}^{-1}(\hat{\mathbf{a}}) = \sum_{i=0}^{n-1} a_i X^i \text{ with } a_i = \frac{1}{n} \sum_{j=0}^{n-1} \hat{a}_j \zeta_n^{-ij}.$$

The powers of the roots of unity used during NTT and NTT^{-1} usually are called “twiddle factors.”

There are two different methods for NTT calculation, Cooley–Tukey butterfly (CT-butterfly) [9] and Gentleman–Sande butterfly (GS-butterfly) [10]. The twisting trick [11] maps $\mathbb{Z}_q[X]/(X^n + 1)$ to $\mathbb{Z}_q[X]/(X^n - 1)$ with $X \mapsto \zeta_{2n}X$ when ζ_{2n} is a primitive $2n$ th root of unity.

B. NTT Framework of Kyber

In the reference implementation of Kyber, NTT is implemented by CT-butterfly, and GS-butterfly implements NTT^{-1} . It avoids the operation of bit reversal. We apply GS-butterfly to implement both NTT and NTT^{-1} . The twisting trick is utilized in our NTT implementation to save the high cost of bit reversal permutation. We illustrate the applied computational framework of Kyber implementation through Fig. 1.

III. CUSTOM INSTRUCTIONS**A. k^2 -Reduction, Montgomery Reduction, and Barrett Reduction**

k^2 -Reduction is a successive composition of two k -reductions (also known as k -RED in [12]). Algorithm 1 is a generalized version of the original k^2 -reduction in [12]. It allows using different values of k (say, k_0 and k_1), while the original one uses the same k value. The negative/positive value of δ impacts the hardware resources since the minus operation in the hardware may require additional calculation for complement values, increasing the cost.

For k^2 -reduction, the range of input value C is $(0, q^2)$. When $\delta = -1$, $r \in [-k2^m + (k2^{i+j}/2^m), k2^{i+j}]$; when $\delta = 1$, $r \in [-k2^j +$

TABLE I
COMPARING THE SYNTHESIS OF REDUCTIONS ON KYBER

	k^2 -reduction		Montgomery	Barrett
	naive	optimized		
range of input	$C \in (0, q^2)$	$C \in (0, q^2)$	$C \in (0, q^2)$	$C \in (0, q^2)$
range of output	$(-q, 2q)$	$(-q, q)$	$(0, 2q)$	$(0, 2q)$
size of major operations	8-bit $\times$ 4-bit; 8-bit $\times$ 4-bit;	8-bit $\times$ 5-bit; 4-bit $\times$ 5-bit;	13-bit $\times$ 12-bit; 13-bit $\times$ 12-bit;	24-bit $\times$ 13-bit; 13-bit $\times$ 12-bit;
GE	359	250	647	2095

$[(k2^{i+j}/2^m), k2^m + k2^{i+j} - k2^j]$. The parameters i , j , and δ affect the overhead of k^2 -reduction in hardware. One can simply assign $i = j = 0$ and $\delta = 1$, which is the naive strategy that previous works, such as [12], used. Alternatively, we can further reduce the hardware overhead by looking for more proper parameters.

Montgomery reduction [13] and Barrett reduction [14] are other popular choices for modular reduction. They are adopted by [6] and [7].

B. Reductions Comparison

We synthesize the above reductions using Design Compiler 2016.03-SP1 with UMC 55-nm technology and measure the circuit size by the metric of gate equivalent (GE). Table I summarizes the comparison.

For k^2 -reduction, there are two multiplications, $k_0 C_{00}$ and $k_1 C_{10}$. When $\delta = 1$, the GE of the naive k^2 -reduction is 359. When $\delta = -1$, the combination of $k_0 = -13$ and $k_1 = -13 \cdot 2^4$ has the least circuit size cost, whose GE is 250. Compared with the naive one, our optimized k^2 -reduction has less GE and leads to a smaller output range. For Montgomery reduction, there are two multiplications too. They are both 13-bit $\times$ 12-bit multiplications. In the case of Barrett reduction, to adhere to the validity constraint, the first multiplication involves a 24-bit $\times$ 13-bit operation, and the complete 37-bit result must be retained. The second multiplication is 13-bit $\times$ 12-bit. According to the above comparison, we use the optimized k^2 -reduction as our basis for the custom instructions.

C. Instruction Set Extensions

We use GS-butterfly for calculations of both NTT and NTT^{-1} , saving more silicon area than adopting CT-butterfly and GS-butterfly on NTT and NTT^{-1} separately. The integrated circuit for the custom GS-butterfly instruction on Kyber is shown in Fig. 2. Thanks to the prime of Kyber, $q = 3329$, which is less than 12 bits, two coefficients, a_0 and a_1 , can be stored simultaneously in one 32-bit register, SRC1. The results `add_res` ($a_0 + a_1$) and `sub_res` ($(a_0 - a_1) \cdot \zeta$) are also packed in one register. The multiplexer `mux` maps an integer to the standard representative.

The modular multiplication, addition, and subtraction in Kyber are implemented by the custom GS-butterfly instruction with appropriate parameters. For example, to apply the custom GS-butterfly instruction as modular multiplication, a_1 is filled with zero, and the factors to be multiplied are put into a_0 and ζ . Then, `sub_res` is the modular multiplication of $a_0 \zeta$.

In order to execute the merging-NTT smoothly, a swap operation is needed. In the merging-NTT, there are no intermediate load or store operations. For instance, (a_0, a_1) and (a_2, a_3) are the coefficient pairs to perform the first layer of NTT. The swapping operation rearranges the coefficients in registers to prepare (a_0, a_2) and (a_1, a_3) for the

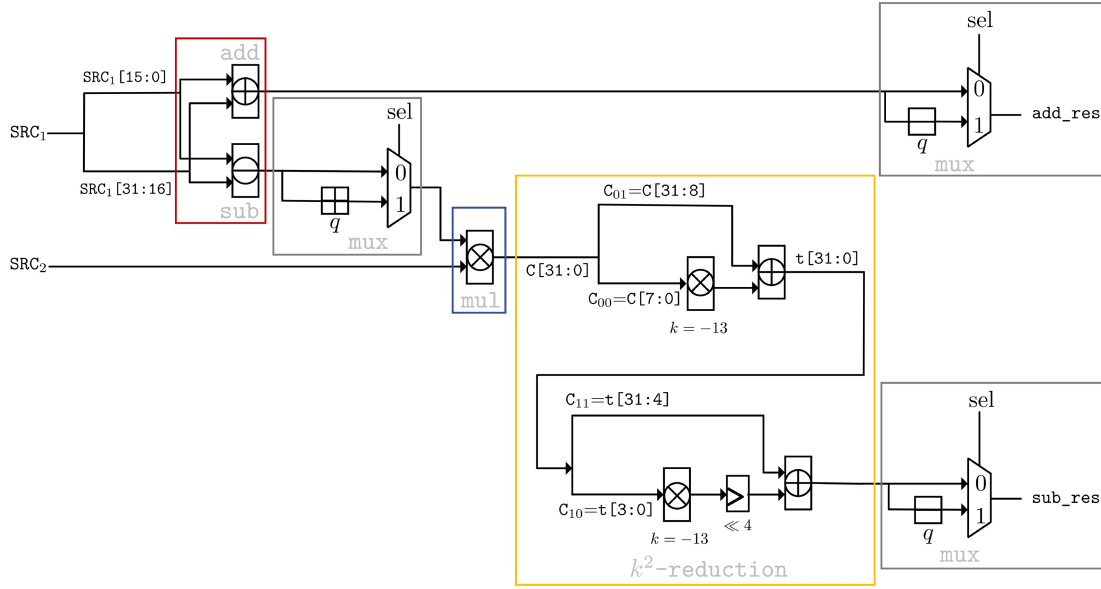


Fig. 2. Circuit diagram for GS-butterfly. The gray boxes represent mux operations. The red boxes are the addition and subtraction. The blue box is a multiplication. The following operation in the orange box is k^2 -reduction.

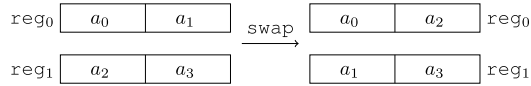


Fig. 3. Swap example.

second NTT layer, as Fig. 3 shows. For further details, please refer to [7].

We integrate GS-butterfly and swap instructions of Kyber as the **R**-type instructions. GS-butterfly instruction is named `butterfly` to accomplish GS-butterfly operation and modular arithmetic. Two swap instructions are `swaphh` and `swapl1`.

IV. EXPERIMENTAL RESULTS

This section presents our experimental results for Kyber. We choose Hummingbird E203 [15] as the basic platform for the evaluation. Our implementation achieves a good tradeoff of speed and hardware overhead.

A. Performance

1) *Polynomial Multiplication*: In this work, we can merge up to four NTT layers depending on 32 registers. Therefore, the seven layers NTTs (NTT and NTT^{-1}) are merged as $4 + 3$. Moreover, multiplication in the NTT domain (`basemul`) is $(\hat{a}_i + \hat{a}_{2i+1}X) \times (\hat{b}_{2i} + \hat{b}_{2i+1}X) \bmod (X^2 - \zeta_n^{2br(i)+1})$, $i = 0, 1, \dots, 127$, a size 2×2 schoolbook multiplication rather than coefficient-wise multiplication.

Table II summarizes the clock cycles required to perform an NTT-based polynomial multiplication over $\mathbb{Z}_{3329}/(X^{256} + 1)$, including NTT, NTT^{-1} , and `basemul`. We evaluate the C-code polynomial multiplication (`C-Ref.`) as the reference implementation of Kyber recommended, the software implementation by RISC-V base ISA (`Assembly Impl.`), as well as the corresponding implementation using our ISEs (`Custom`). We add recent results from [6] and [7] on RISC-V cores as benchmarks.

Our implementation obtains a higher performance gain compared with [6]. However, our base multiplication is a bit slower, which is the side effect of applying the custom GS-butterfly instruction to calculate

TABLE II
CYCLE COUNTS FOR THE POLYNOMIAL MULTIPLICATION
OVER $\mathbb{Z}_{3329}/(X^{256} + 1)$

	Impl.	NTT		basemul	NTT^{-1}	Total
		twist	cyclic -NTT			
Our work	Custom	1251	2938	3257	3481	10927
	Assembly Impl.	4189		8140	12936	31973
	C-Ref.	24670		9263	34911	68844
[6]	Custom	6868		2395	6367	17216
	Assembly Impl.	8595		4050	9427	23658
[7]	Custom	1935		N/A	1930	3865

all modular arithmetic. The minor decrease in speed is tolerable as it does not diminish our advantage in polynomial multiplication. The cycle count of designs in [7] is much smaller than that of [6] and ours. It is because that [7] provided high-performance ISEs at the cost of more circuit resources.

2) *Kyber Implementation*: There are many recent accelerated implementations of Kyber; see [16], [17], and [18] for an incomplete list. It should be noted that those results may not be directly compared with ours because of different optimization goals. They implement a complete NTT to speed up the implementation at the cost of more hardware resources. For example, 644 LUTs, 260 FFs, and 28 RAMs are used in the recent study [16], while we only need 93 LUTs and no FFs and RAMs.

We mainly consider the tightly coupled designs in [6] and [7] as the benchmarks for comparison. The results of Kyber implementations are summarized in Table III. In [6], there is no result of Kyber768. A simple assembly code evaluates the Keccak permutation in `Assembly Impl.` and `Custom`. Our `Custom` is more efficient than `C-Ref.` and `Assembly Impl.`. The results also show that our custom instructions have higher efficiency than [6]. Certainly, the implementation with all optimizations presented in [7] is quite fast, as seen in `opt.all`. Our implementation is a better tradeoff, as [7] incurred greater hardware overhead in integrating accelerators.

TABLE III
PERFORMANCE (IN KILO CYCLES) OF KYBER.KEM IMPLEMENTATIONS
FOR KEY GENERATION (K), ENCAPSULATION (E),
AND DECAPSULATION (D)

		Impl.	K	E	D
Kyber -512	Our work	C-Ref.	1 197	1 568	1 546
		Assembly Impl.	703	910	897
		Custom	622	785	713
	[6]	C-Ref.	928	1 301	1 345
		Assembly Impl.	738	1 009	923
		Custom	710	971	870
	[7]	C-Ref.	1 138	1 548	1 526
		opt.all	151	194	205
Kyber -768	Our work	C-Ref.	1 949	2 506	2 465
		Assembly Impl.	1 143	1 452	1 426
		Custom	988	1 237	1 133
	[7]	C-Ref.	2 103	2 626	2 574
		opt.all	274	326	341
		C-Ref.	3 068	3 753	3 688
Kyber -1024	Our work	Assembly Impl.	1 773	2 151	2 117
		Custom	1 543	1 851	1 719
		C-Ref.	2 745	3 319	3 356
	[6]	Assembly Impl.	2 274	2 703	2 535
		Custom	2 203	2 619	2 429
		C-Ref.	3 379	4 025	3 950
	[7]	opt.all	350	406	425
		C-Ref.	3 068	3 753	3 688
		Assembly Impl.	1 773	2 151	2 117

B. Circuit Resources

We synthesize the E203 embedded with our instructions for FPGA and ASIC. Our evaluation involves three different E203 cores. “E203 original” (orig.) is a default E203 core. “custom” is the core embedded in our new ISEs. The instructions in [6] replace a full general-purpose multiplier to achieve very compact implementations, and this strategy is also compatible with our instructions. Concretely, we also provide a custom core without a general-purpose multiplier “custom-nomul.”

We choose an Artix-7 35T FPGA evaluation and use the Xilinx Vivado 2021.1 toolchain for synthesis. It is one of the reference platforms in the NIST standardization process. The works of [6] and [7] are on Artix-7 and Zynq-7000, respectively, where Zynq-7000 is based on Artix-7 or Kintex-7, and Kintex-7 uses the same FPGA resources as Artix-7, and their FPGA building blocks are functionally identical.

Table IV shows the results for device utilization on FPGA. We consider the relative circuit resources of the custom cores from the original one. The results show that the overhead of our ISEs is quite small. “custom” uses 93 additional LUTs and 1 DSP from “orig.” and no additional FFs or RAMs. The absolute frequency is related to the specific RISC-V core and platform. The hardware complexity of ISEs determines the decrease in the frequency. For example, [7] created 29 instructions to speed up operations, including modular arithmetic, butterflies, and bit-reversal, with the maximum frequency reduced by 42.9% (from 79.66 to 45.47 MHz). We create custom instructions in a relatively simple circuit with a 6.8% frequency decrease (from 35.3 to 32.9 MHz).

TABLE IV
XILINX ARTIX-35T DEVICE UTILIZATION

		LUTs	FFs (Registers)	RAMs	DSPs
Our work	orig. (E203)	9 521	6 669	32	4
	custom	orig.+93	orig.+0	orig.+0	orig.+1
	custom-nomul	orig.-311	orig.-0	orig.-0	orig.-2
[6]	orig. (VexRiscv)	1 738	1 599	34	4
	custom	orig.+104	orig.+33	orig.+0	orig.+1
	custom-nomul	orig.-242	orig.-301	orig.-0	orig.-2
[7]	orig. (PULPino)	14 715	9 583	32	6
	custom-opt.arith.	orig.+886	orig.+618	orig.+1	orig.+26
	custom-opt.all.	orig.+9 232	orig.+1 264	orig.+0	orig.+13

TABLE V
AREA RESULTS OF ASIC SYNTHESIS (UMC 55 NM)

		GE	Combinational cells	Sequential cells
Our work	orig. (E203)	18 554	12 763	313
	custom	orig. +799	orig. +442	orig. +55
	custom-nomul	orig. -5 203	orig. -2 818	orig. -5
[7]		Cell count	Combinational cells	Sequential cells
	orig. (PULPino)	36173	78676	92304
	custom-opt.all.	orig. +21 240	orig. +64 522	orig. +9 969

TABLE VI
POWER RESULTS OF ASIC SYNTHESIS (UMC 55 NM)

		Leakage(mW)	Internal(mW)	Switching(mW)	Tot. Power(mW)
Our work	orig.(E203)	8.3027e-03	0.1444	0.1566	0.3093
	custom	orig.+0.2767e-03	orig.+0.0034	orig.+0.0044	orig.+0.0081
	custom-nomul	orig.-1.6894e-03	orig.-0.0058	orig.-0.0076	orig.-0.0151
[7]	orig.	2.9600e-01	1.3800	0.3960	2.0700
	custom-opt.all	orig.+0.0200e-01	orig.+0.2100	orig.+0.2930	orig.+0.5000

For the ASIC evaluation, we choose UMC 55-nm technology and use the Design Compiler 2016.03-SP1 toolchain for synthesis. Table V shows the number of logic cells and the area consumption of our work and [7]. In [6], there is no result of area consumption in ASIC technology. “custom-nomul” saves more silicon area in ASIC synthesis compared to FPGA synthesis because the two DSPs saved in FPGA correspond to many GEs in ASIC. In summary, the results of ASIC synthesis are consistent with those of the FPGA, and our work has less area consumption than [7]. Moreover, Table VI summarizes the power consumption. Our ISE consumes significantly less power compared with [7]. For example, in terms of the total power, the “custom” core additionally consumes 0.0081 mW, while the previous “custom-opt.all” core in work [7] additionally consumes 0.5 mW. The low power consumption property makes our design more compatible to the low-power scenarios such as the radio frequency identification (RFID).

V. CONCLUSION

This brief presents a hardware design of the modular reduction based on the optimized k^2 -reduction. Our design is notably compact with additional 93 LUTs. We customize GS-butterfly instruction based on the optimized k^2 -reduction. With swap instructions, we accelerate the coefficients rearrangement in NTT. Finally, we implement the new ISEs in the Hummingbird E203 core. Compared with the baseline implementations, the performance of our ISEs achieves a significant tradeoff between speed and hardware overhead.

REFERENCES

- [1] R. Avanzi et al., “CRYSTALS-Kyber algorithm specifications and supporting documentation (version 3.0, post-quantum cryptography standardization project),” Nat. Inst. Stand. Technol., Gaithersburg, MD, USA, document, 2020. [Online]. Available: <https://csrc.nist.gov/CSRC/media/Projects/postquantum-cryptography/documents/round-3/submissions/Kyber-Round3.zip>
- [2] (Nat. Inst. Stand. Technol., Gaithersburg, MD, USA). *PQC Standardization Process: Announcing Four Candidates to be Standardized, Plus Fourth Round Candidates*. (2022). [Online]. Available: <https://csrc.nist.gov/News/2022/pqc-candidates-to-be-standardized-and-round-4>

- [3] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," Electr. Eng. Comput. Sci. Dept., Univ. California, Berkeley, CA, USA, Rep. UCB/EECS-2014-146, Aug. 2014.
- [4] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, "Sapphire: A configurable Crypto-processor for post-quantum lattice-based protocols," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2019, no. 4, pp. 17–61, 2019.
- [5] W. Wang, S. Tian, B. Jungk, N. Bindel, P. Longa, and J. Szefer, "Parameterized hardware accelerators for lattice-based cryptography and their application to the HW/SW co-design of qTESLA," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 3, pp. 269–306, 2020.
- [6] E. Alkim, H. Evkan, N. Lahr, R. Niederhagen, and R. Petri, "ISA extensions for finite field arithmetic—Accelerating Kyber and Newhope on RISC-V," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 3, pp. 219–242, 2020.
- [7] T. Fritzmann, G. Sigl, and J. Sepúlveda, "RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 4, pp. 239–280, 2020.
- [8] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and benchmarking NIST PQC on ARM cortex-M4," Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2019/844, 2019.
- [9] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.
- [10] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proc. AFIPS*, 1966, pp. 563–578.
- [11] D. J. Bernstein, "Multidigit multiplication for mathematicians," *Adv. Appl. Math.*, to be published.
- [12] P. Longa and M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," in *Proc. 15th Int. Conf.*, Milan, Italy, vol. 10052, 2016, pp. 124–139.
- [13] P. L. Montgomery, "Modular multiplication without trial division," *Math. Comput.*, vol. 44, no. 170, pp. 519–521, 1985.
- [14] P. Barrett, "Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor," in *Proc. Conf. Theory Appl. Cryptogr. Techn.*, vol. 263, Santa Barbara, CA, USA, 1986, pp. 311–323.
- [15] "Open-source Hummingbirdv2 E203 RISC-V processor core and SoC," github.com. [Online]. Available: https://github.com/riscv-mcu/e203_hbirdv2
- [16] R. Salarifard and H. Soleimany, "Efficient accelerator for NTT-based polynomial multiplication," Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2023/686, 2023.
- [17] Z. Ni, A. Khalid, D. Kundi, M. O'Neill, and W. Liu, "Efficient pipelining exploration for a high-performance CRYSTALS-Kyber accelerator," Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2022/1093, 2022.
- [18] M. Bisheh-Niasar, R. Azarderakhsh, and M. M. Kermani, "Instruction-set accelerated implementation of CRYSTALS-Kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4648–4659, Nov. 2021.